8_Databases

Wednesday, 24 September 2025 4:07 pm

8.1_database_conce...

8.1 Database Concepts

Created	@September 24, 2025 2:37 PM
Tags	

File-Based Systems

- **Definition**: Data is stored in one or more separate computer files. Each application has its own structure, rules, and processing code.
- **Characteristics**:
  - Records may be fixed or variable in length.
  - Programs must match the exact record structure; if structure changes, programs must be rewritten.

Limitations

1. **Data Redundancy**: Same data stored multiple times across files (e.g., staff name in payroll and sales files).
2. **Data Inconsistency**: Data updated in one file may not be updated in another, leading to conflicting information.
3. **Data Dependency**: File formats and structures are tied to specific applications. Any change requires rewriting dependent programs.
4. **Wasted Storage Space**: Duplication across separate files.
5. **Lack of Data Privacy**: If one file is accessed, all of its data is visible.
6. **Limited Flexibility**: Enquiries depend on file structures and the applications written for them.

Relational Databases

- **Definition**: A structured collection of records where data is stored in tables (rows = records, columns = fields), linked together through keys and relationships.
- **Key Concept**: Internal pointers (primary/foreign keys) connect related data across tables.

Advantages Over File-Based Approach

1. **Reduced Data Redundancy**: Data stored once, reused across multiple applications.
2. **Consistency**: Data changed in one place is reflected everywhere.
3. **Data Independence**: Enquiries and applications are not tied to rigid file structures.
4. **Efficient Storage**: Avoids duplication, saving space.
5. **Improved Integrity**: Referential integrity rules ensure accurate and consistent data.
6. **Improved Privacy**: User access rights restrict who can view or change data.

Benefits of Relational Databases

- **Reduced Data Dependency**: Applications continue to work even if the database structure changes.
- **Improved Data Privacy**: Role-based access ensures users only see authorized data.
- **Support for Ad-hoc Queries**: SQL enables flexible data retrieval without restructuring.
- **Better Data Integrity**: Accuracy and consistency enforced across the database.
- **Program–Data Independence**: Applications and data structure evolve separately.

- **Better Data Integrity**: Accuracy and consistency enforced across the database.
- **Program–Data Independence**: Applications and data structure evolve separately.

Features of Relational Databases

1. **Linked Tables**

   - Multiple tables connected via primary and foreign keys.
   - Referential integrity ensures relationships remain valid.
2. **Complex Queries**
   - SQL allows precise retrieval of specific information.
   - Supports complex filtering, sorting, and summarization.
3. **Access Rights**
   - Users assigned different privileges (read/write/view).
   - Example: Sales staff see only customer contacts, not financial data.
4. **Data Views**
   - Tailored displays of data for different users.
   - Enhances privacy and usability, as each user only sees relevant information.

Key Terminology

Database and DBMS

- **Database**: A structured collection of data stored in tables. Each table holds data about one type of entity (e.g., Students, Classes).
- **DBMS (Database Management System)**: Software that manages databases, handling cataloguing, retrieval, updates, and queries (e.g., MySQL, Oracle, Access).

Entities, Attributes, Fields, and Records

- **Entity**: A real-world object/concept stored in the database (e.g., `Student`, `Book`, `Class`).
- **Attributes**: Characteristics of an entity, stored as columns (e.g., `Name`, `DateOfBirth`).
- **Field**: A single column in a table holding data of one attribute.
- **Record (Tuple)**: A row in the table representing one instance of the entity (e.g., one student).

📌 Example – *Student table*:

StudentID	FirstName	SecondName	DateOfBirth	ClassID
S1276	Noor	Baig	22/09/2010	7A
S1277	Ahmed	Sayed	11/06/2010	7B

Keys

- **Candidate Key**: Attribute(s) that could uniquely identify each record. A table can have multiple candidate keys.
- **Primary Key**: One candidate key chosen as the main unique identifier (e.g., `StudentID`). Must never be duplicated or null.
- **Secondary Key**: Any candidate key not chosen as the primary key, useful for indexing/search (e.g., `DateOfBirth` or `Name`).
- **Foreign Key**: An attribute in one table that refers to the primary key in another table, creating relationships (e.g., `ClassID` in the `Student` table links to the `Class` table).

📌 Example – *Class table*:

ClassID	TeacherName	Location

📌 Example – *Class table*:

ClassID	TeacherName	Location
7A	Mr Khan	Floor 2 Room 3
7B	Miss Malik	Floor 2 Room 4

Here, `ClassID` is:

- **Primary Key** in `Class` table.
- **Foreign Key** in `Student` table.

Relationships

- **One-to-One (1:1)**: One record in Table A matches exactly one record in Table B.
- **One-to-Many (1:m)**: A primary key in one table is referenced many times in another (e.g., one `Class` has many `Students`).
- **Many-to-One (m:1)**: Inverse of above; many students belong to one class.
- **Many-to-Many (m:m)**: Records in both tables can link to multiple records in the other (requires a junction table).

📌 Example: `Student–Class` → many-to-one. One class (7A) can have many students, but each student belongs to only one class.

Referential Integrity

- Ensures foreign keys always match a valid primary key in the related table.
- Prevents orphan records (e.g., a `Student` assigned to a non-existent `Class`).

Indexing

- Indexes are created on attributes to speed up searches.
- Example: Indexing `SecondName` in `Student` table allows faster alphabetical listings of students.

Entity–Relationship (E-R) Diagrams

- An **Entity–Relationship (E-R) diagram** is a visual tool for database design.
- It shows:
  - **Entities** (the main objects, e.g., Student, Class).
  - **Attributes** (the properties of entities, e.g., Name, Date of Birth).
  - **Relationships** (how entities are linked, e.g., "one class has many students").
- Helps both developers and non-technical users easily understand the structure of the database.

Entities and Attributes

- **Entity** = a table in the database (e.g., `Student`, `Class`).
- **Attributes** = fields (columns) that describe each entity.
  - Example:
    - `Student` → StudentID, FirstName, SecondName, DateOfBirth, ClassID.
    - `Class` → ClassID, TeacherName, Location.
- **Primary Key** uniquely identifies each entity instance (e.g., StudentID, ClassID).

Relationships

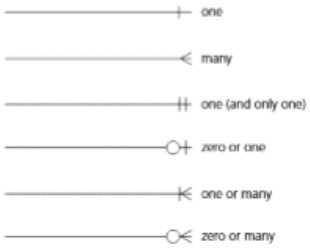
- Relationships describe how entities are connected.

Relationships

- Relationships describe how entities are connected.
- Example: In a school database:
  - **One class has many students** → Relationship between `Class` and `Student`.
  - `ClassID` is the primary key in the Class table, and a foreign key in the Student table.

Cardinality

- **Cardinality** = the number of instances in one entity that can be associated with instances in another entity.
- Types of relationships:
  1. **One-to-One (1:1)**
     - Each entity in Table A matches exactly one in Table B.
     - Example: One employee ↔ one desk.
  2. **One-to-Many (1:m)**
     - One entity in Table A can be linked to many in Table B.
     - Example: One class ↔ many students.
  3. **Many-to-One (m:1)**
     - Many records in Table A belong to one record in Table B.
     - Example: Many students ↔ one class.
  4. **Many-to-Many (m:m)**
     - Records in both tables can be linked to many in the other (implemented with a linking table).
     - Example: Students ↔ Courses (each student can take many courses, each course has many students)



Q1 The shop owner creates a relational database called FIXIT.

The database stores data about the customers and the devices for repair.

Some devices need new parts that are ordered from suppliers.

The database FIXIT is designed to include the following tables:

PART(PartID, Description, Price, SupplierID)

CUSTOMER(CustomerID, FirstName, LastName, ContactNumber)

REPAIR(RepairNumber, StartDate, EndDate, CustomerID, Device)

REPAIR_PART(PartID, RepairNumber, Quantity)

Complete the entity-relationship (E-R) diagram for the given tables.

REPAIR_PART PART

REPAIR CUSTOMER

[3]

Q2 A shop manager has designed a relational database to store customer orders.

The database will have the following tables:

CUSTOMER(CustomerID, FirstName, LastName, Town)

SHOP_ORDER(OrderNo, CustomerID, OrderDate)

SUPPLIER(SupplierID, EmailAddress, TelephoneNumber)

ITEM(ItemNumber, SupplierID, Description, Price)

ORDER_ITEM(ItemNumber, OrderNo, Quantity)

Complete the entity-relationship (E-R) diagram for the relational database.

SHOP_ORDER CUSTOMER

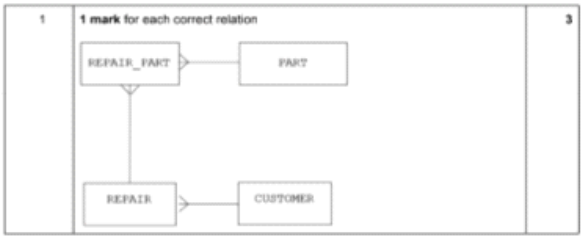
ORDER_ITEM SUPPLIER

ITEM

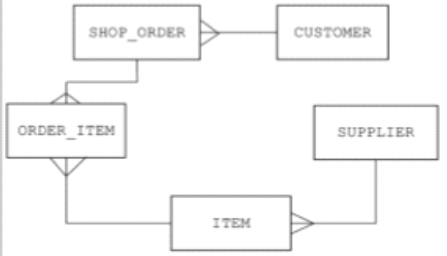
[3]

(Q2 a) 9618/11/O/N/23

[3]



Question	Answer	Marks
2	**1 mark** for each correct relationship or relationships • 1:M between CUSTOMER and SHOP_ORDER • 1:M between SUPPLIER and ITEM • 1:M between SHOP_ORDER and ORDER_ITEM **and** M:1 between ORDER_ITEM and ITEM	3



Normalisation in Databases

Normalisation is the process of organising a relational database to:

- **Reduce redundancy** – avoid repeating the same data in multiple places.
- **Maintain integrity** – ensure data remains consistent and accurate across the database.
- **Simplify updates** – changes made once will reflect everywhere, reducing anomalies.
- **Support efficient querying** – smaller, well-structured tables are easier to manage.

Without normalisation, tables grow large, include duplicated data, and cause problems when inserting, updating, or deleting records (known as anomalies).

First Normal Form (1NF)

Rules:

1. **No Repeated Groups/Attributes**: Each column must contain atomic (indivisible) values — no sets, lists, or repeating groups inside a single field.
   - ❌ Example: A single "Subjects" column holding "Maths, Science, History".
   - ✅ Instead: Each subject goes into its own row.
2. **Attributes Must Be Atomic**: Data should be broken down to the smallest meaningful unit.
   - ❌ Example: A "Full Name" field.
   - ✅ Instead: "First Name" and "Last Name" fields.
3. **No Duplicate Rows**: Every row must be unique, typically enforced by a **primary key**.

Example:

Unnormalised student table (subjects repeated inside one field):

StudentID	Name	Subjects
S1276	Noor Baig	Maths, History, Geography

After 1NF (separate rows for each subject):

StudentID	First Name	Last Name	Subject

S1276	Noor	Baig	Maths, History, Geography

After 1NF (separate rows for each subject):

StudentID	First Name	Last Name	Subject
S1276	Noor	Baig	Maths
S1276	Noor	Baig	History
S1276	Noor	Baig	Geography

Second Normal Form (2NF)

Rules:

1. Must already satisfy **1NF**.
2. **No Partial Dependency**: Every non-key attribute must depend on the **whole primary key** — not just part of it.
   - This rule is important when the table has a **composite key** (a key made of more than one field).

Example:

Consider a table with a composite key (StudentID + SubjectName).

If "SubjectTeacher" only depends on "SubjectName" (not the whole key), then we have a **partial dependency**.

Split into two tables:

- **STUDENTSUBJECT(StudentID, SubjectName)**
- **SUBJECT(SubjectName, SubjectTeacher)**

Now each non-key attribute depends fully on its primary key.

❌ Table not in 2NF

Suppose we track **Orders** with a composite key `(OrderID, ProductID)`:

OrderID	ProductID	ProductName	CustomerName
O1	P1	Laptop	Ali Khan
O1	P2	Mouse	Ali Khan
O2	P1	Laptop	Hira Ali

Problem:

- Composite key = `(OrderID, ProductID)`.
- **ProductName** depends only on `ProductID` (not whole key).
- **CustomerName** depends only on `OrderID`.

  👉 Partial dependencies → violates 2NF.

✅ In 2NF (remove partial dependencies)

ORDER

OrderID	CustomerName
O1	Ali Khan
O2	Hira Ali

PRODUCT

ProductID	ProductName
P1	Laptop
P2	Mouse

ORDER-DETAIL

OrderID	ProductID

P2	Mouse

ORDER-DETAIL

OrderID	ProductID
O1	P1
O1	P2
O2	P1

Third Normal Form (3NF)

Rules:

1. Must already satisfy **2NF**.
2. **No Transitive Dependency**: Non-key attributes must not depend on other non-key attributes. They should depend **only** on the primary key.

Example:

❌ Table not in 3NF

Suppose we track **Employees**:

EmpID	EmpName	DeptID	DeptName	DeptLocation
E1	Ali Khan	D1	Sales	Floor 2
E2	Hira Ali	D2	HR	Floor 3
E3	Ahmed Raza	D1	Sales	Floor 2

Problem:

- Primary key = `EmpID`.
- **DeptName** depends on DeptID (not directly on EmpID).
- **DeptLocation** depends on DeptName → transitive dependency.

  👉 Violates 3NF.

✅ In 3NF (remove transitive dependencies)

EMPLOYEE

EmpID	EmpName	DeptID
E1	Ali Khan	D1
E2	Hira Ali	D2
E3	Ahmed Raza	D1

DEPARTMNET

DeptID	DeptName	DeptLocation
D1	Sales	Floor 2
D2	HR	Floor 3

Summary of Normal Forms

Normal Form	What It Removes	Key Idea
1NF	Repeating groups	Fields are atomic; no duplicate rows
2NF	Partial dependencies	Every non-key field depends on the whole primary key
3NF	Transitive dependencies	Non-key fields depend only on the primary key

1NF	Repeating groups	Fields are atomic; no duplicate rows
2NF	Partial dependencies	Every non-key field depends on the whole primary key
3NF	Transitive dependencies	Non-key fields depend only on the primary key

Example of Why Normalisation Matters

Unnormalised school table:

- Each time a student is added, their teacher's details are repeated.
- If a teacher changes rooms, updates must be made in every student row.
- If all students in Class 7B leave, all data about 7B is lost.

By normalising to **3NF**, the data is split into logical tables — Students, Teachers, Classes, Subjects — making updates efficient and safe.

EXAMPLE

A raw table often includes **repeated groups** and redundant data.

StudentID	StudentName	ClassID	ClassRoom	TeacherName	Subjects
S1	Adeel Khan	7A	Room 3	Mr Khan	Maths, Science, History
S2	Hira Ali	7B	Room 4	Miss Malik	Maths, English
S3	Ali Raza	7A	Room 3	Mr Khan	Maths, Science

Problems:

- **Repeating groups** (Subjects listed together).
- **Redundancy** (Mr Khan and Room 3 repeated for every student in Class 7A).
- **Hard to update** (If Mr Khan changes class, must update multiple rows).

First Normal Form (1NF)

👉 Remove repeating groups → each subject gets its own row.

👉 Ensure all attributes are **atomic** (indivisible).

StudentID	StudentName	ClassID	ClassRoom	TeacherName	Subject
S1	Adeel Khan	7A	Room 3	Mr Khan	Maths
S1	Adeel Khan	7A	Room 3	Mr Khan	Science
S1	Adeel Khan	7A	Room 3	Mr Khan	History
S2	Hira Ali	7B	Room 4	Miss Malik	Maths
S2	Hira Ali	7B	Room 4	Miss Malik	English
S3	Ali Raza	7A	Room 3	Mr Khan	Maths
S3	Ali Raza	7A	Room 3	Mr Khan	Science

Fixes:

- Split multiple subjects into separate rows.
- All fields now hold **single values**.

Second Normal Form (2NF)

👉 Must already be in 1NF.

👉 Eliminate **partial dependency** (fields depending only on part of a composite key).

In the current table, the composite key is **(StudentID, Subject)**.

But notice:

👉 Eliminate **partial dependency** (fields depending only on part of a composite key).

In the current table, the composite key is **(StudentID, Subject)**.

But notice:

- **ClassRoom** and **TeacherName** depend only on ClassID, not on StudentID+Subject.

Split into new tables:

STUDENT

StudentID	StudentName	ClassID
S1	Adeel Khan	7A
S2	Hira Ali	7B
S3	Ali Raza	7A

CLASS

ClassID	ClassRoom	TeacherName
7A	Room 3	Mr Khan
7B	Room 4	Miss Malik

STUDENT_SUBJECT

StudentID	Subject
S1	Maths
S1	Science
S1	History
S2	Maths
S2	English
S3	Maths
S3	Science

Fixes:

- Now **ClassRoom/TeacherName** only stored once per class (not repeated per student).
- No **partial dependency** remains.

Third Normal Form (3NF)

👉 Must already be in 2NF.

👉 Eliminate **transitive dependency** (non-key fields depending on other non-key fields).

In the CLASS table, **TeacherName** → determines things like TeacherDOB, Address (if we stored them). This is a **transitive dependency**.

Split further:

STUDENT

StudentID	StudentName	ClassID
S1	Adeel Khan	7A
S2	Hira Ali	7B
S3	Ali Raza	7A

CLASS

ClassID	ClassRoom	TeacherID

CLASS

ClassID	ClassRoom	TeacherID
7A	Room 3	T1
7B	Room 4	T2

TEACHER

TeacherID	TeacherName	DOB	Address
T1	Mr Khan	03/27/1985	School House 1
T2	Miss Malik	12/14/1988	School House 2

STUDENT_SUBJECT

StudentID	Subject
S1	Maths
S1	Science
S1	History
S2	Maths
S2	English
S3	Maths
S3	Science

Summary of Fixes

Stage	Problem	Solution
UNF	Repeated groups (Subjects in one cell)	Split into rows → 1NF
1NF	Partial dependency (ClassRoom, TeacherName depend only on ClassID)	Split into separate tables → 2NF
2NF	Transitive dependency (Teacher details depend on TeacherName, not directly on ClassID)	Create TEACHER table, use TeacherID → 3NF
3NF	✓ Fully normalised	All non-key fields depend only on the key, the whole key, and nothing but the key